SafeBoxForest: Rapid C-space Box Cover with Lifelong Envelope Caching for GCS Planning

TIAN, Yu and Ren, Hongliang*
Department of Electronic Engineering, The Chinese University of Hong Kong,
Shatin, N.T., Hong Kong SAR, China
{ty1997, hlren}@ee.cuhk.edu.hk

***Abstract*—Graph of Convex Sets (GCS) is a state-of-the-art framework for convex-optimization-based motion planning that requires convex free-space regions as input. Existing IRIS-family region generators either lack collision-safety certificates or require minutes per region, and none supports incremental updates when the scene changes. We present *SafeBoxForest* (SBF), a certified region generator for GCS. SBF introduces a *Lifelong Envelope Cache Tree* (LECT) that maps C-space boxes to certified workspace axis-aligned bounding box (AABB) envelopes via *interval forward kinematics* (FK). Because envelopes depend only on robot kinematics, they are computed once on demand and cached for the robot's lifetime. Given an obstacle scene, SBF queries the LECT and grows a collision-free box forest via wavefront expansion. Greedy coarsening merges fine boxes into larger certified regions that map directly to a GCS graph. When obstacles change, only the affected boxes are regrown. On a 7-DOF KUKA IIWA14 manipulator, SBF generates certified regions 86–$123\times$ faster than IRIS-NP with a 98% GCS planning success rate. Incremental regrowth takes just $7\,\mathrm{ms}$ ($1/150$ of a cold rebuild).**

I. Introduction

Graph of Convex Sets (GCS) [1] formulates motion planning as optimization over a graph of convex free-space regions. It produces high-quality trajectories via convex relaxation. However, planning quality depends on the upstream *region generation* step: how to quickly produce certified collision-free convex regions. In assembly lines and human–robot collaboration, obstacles change frequently while the robot remains the same, so region generators must support incremental updates rather than full rebuilds. Existing IRIS-family methods do not meet this need. IRIS-NP [2] lacks formal collision-safety certificates. C-IRIS [3] provides certificates but requires minutes per region. No existing method supports persistence or cross-scene reuse. SBF targets applications where formal collision-free guarantees are mandatory (subject to the geometry-enclosure assumption stated in section III-A). It trades individual-region volume for certified safety and orders-of-magnitude speed advantage.

For a serial manipulator, given a C-space box, one can compute a workspace AABB for each link that encloses the link geometry over all configurations in that box. We call this AABB a *link-AABB envelope* (fig. 1a–b). The mapping from C-space box to link-AABB vector depends *only* on robot kinematics, *not* on obstacles. This observation enables a lazy caching design. A KD-tree called the LECT lazily computes and caches AABB envelopes for C-space cells on demand. Each envelope is computed once and reused permanently across all scenes. Given a specific scene, SBF queries the LECT for cached AABBs and grows a collision-free box forest. The forest maps directly to a GCS graph. Based on this observation, SBF makes the following contributions:

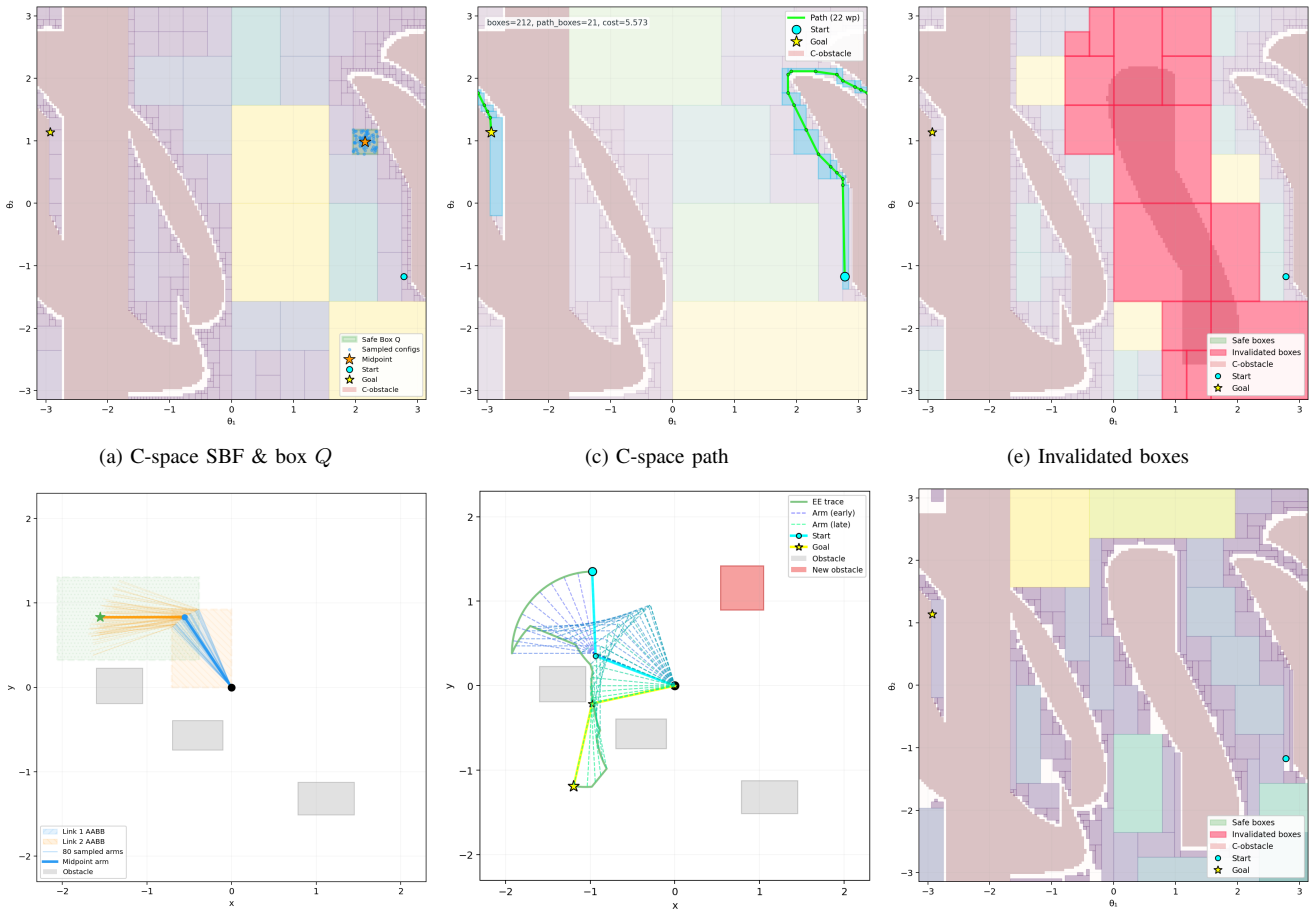
1) **Certified lifelong caching** (sections III-A and III-B): The LECT maps C-space boxes to certified workspace AABB envelopes via interval-FK. Each envelope is computed on demand and cached permanently. Because envelopes depend only on robot kinematics, the cache is reusable across all scenes for the robot's lifetime.
2) **Rapid certified region generation** (sections IV-A, IV-B and IV-D): Starting from seed configurations, boxes expand outward via wavefront expansion. Bridge paving connects isolated components. A tree-cached two-stage hull check then merges thousands of fine boxes into hundreds of certified regions that map directly to a GCS graph.
3) **Strong cross-scene reusability** (section IV-C): When obstacles change, targeted regrowth repairs only the affected boxes, reducing update time from seconds to milliseconds.

Comprehensive experiments on a 7-DOF KUKA IIWA14 manipulator (section V) validate SBF's region-generation efficiency, end-to-end GCS planning performance, and incremental-update efficiency under obstacle perturbation.

II. Related Work

A. GCS Planning and Region Generation

GCS [1] casts motion planning as a mixed-integer convex program over a graph of convex sets. Tight convex relaxations yield globally optimal trajectories. IRIS [4] alternates between inscribed ellipsoids and separating hyperplanes in workspace but does not directly produce C-space regions. IRIS-NP [2] grows C-space polytopes via nonlinear seed-growing, achieving high volume but without formal collision-safety certificates. IRIS-ZO and IRIS-NP2 [5] apply zeroth-order and improved nonlinear optimization to accelerate region generation, yet remain non-certified. C-IRIS [3] provides sum-of-squares (SOS) polynomial certificates but requires minutes per region in high-dimensional settings. No existing method supports persistence or cross-scene reuse.



(a) C-space SBF & box Q (c) C-space path (e) Invalidated boxes

(b) Workspace AABBs of Q (d) Workspace path (f) After regrowth

Fig. 1. SBF visualization on a planar 2-link manipulator. (a, b) Certified box forest in C-space and the workspace AABB envelopes of highlighted box Q. (c, d) GCS planned path through the coarsened box forest in C-space and the corresponding workspace trajectory. (e, f) After a new obstacle is added, invalidated boxes (red) are identified and targeted regrowth repairs the region.

B. C-space Decomposition and Cell Methods

Exact cell decomposition [6] is theoretically complete but intractable in high dimensions. Approximate cell decomposition [7] employs octree/KD-tree recursive splitting but lacks safety certification. Approximate convex decomposition [8] primarily targets workspace geometry and has not been extended to C-space certification. SBF bridges the gap between approximate decomposition and certified safety by combining interval-FK certificates with hierarchical cell refinement.

C. Interval Arithmetic in Robotics

Interval analysis provides rigorous enclosures of function ranges [9], [10]. Interval-FK has been used for workspace reachability analysis [11] and parallel robot calibration [12]. To the best of our knowledge, SBF is the first system to apply interval-FK for C-space region certification in the context of GCS planning.

D. Sampling-Based Motion Planning

Rapidly-exploring Random Tree (RRT) [13] grows a search tree by repeatedly sampling a random configuration and extending the nearest node toward it. RRT-Connect [14] accelerates convergence by growing two trees from start and goal simultaneously. RRT* [15] adds asymptotic optimality via rewiring. All RRT-family planners explore C-space with point samples. They produce a tree of configurations, not convex regions. SBF's forest growth borrows the wavefront expansion idea from RRT: a seed configuration is analogous to an RRT root, and face sampling extends the frontier outward. The key difference is that SBF expands certified *boxes* rather than point samples. Each box carries an interval-FK safety certificate and directly serves as a GCS node.

E. Multi-Query and Persistent Planning

Probabilistic Roadmap (PRM) [16] builds a persistent roadmap that supports multi-query planning but provides neither convex regions nor collision-safety certificates. Experience Graphs [17] reuse historical paths but do not generate free-space regions. SBF uniquely combines certified convex region generation, persistence, and incremental updates in a single framework.

III. Lifelong Certified Envelope Cache Tree

This section first presents the certification problem and interval-FK, which computes certified AABB envelopes for C-space boxes. The LECT then organizes these envelopes in

a lazy KD-tree cache that is scene-independent and reusable across the robot's lifetime.

A. Certified Envelope via Interval-FK

Consider a D-DOF serial manipulator with L collision-relevant links. Given a C-space box $B \subset \mathbb{R}^D$ and a set of obstacle AABBs $\mathcal{O}$ in the workspace, the goal is to determine whether $B \subseteq \mathcal{C}_{\text{free}}$, *i.e.*, the robot is collision-free for every $\boldsymbol{q} \in B$.

Link-AABB envelope. For link ℓ and C-space box B, the *link-AABB envelope* $\text{AABB}_\ell(B)$ is a workspace AABB satisfying

$$\text{AABB}_\ell(B) \supseteq \bigcup_{\boldsymbol{q} \in B} \text{geom}_\ell\big(\text{FK}(\boldsymbol{q})\big). \tag{1}$$

Each link is modeled as a line segment connecting two adjacent joint origins, inflated by a collision radius r_ℓ that conservatively encloses the true mesh geometry. We assume that this inflated capsule always contains the true link geometry (assumption 1).

Assumption 1 (Geometry Enclosure)**.** *For each link ℓ and all $\boldsymbol{q} \in \mathcal{C}$, $\text{geom}_\ell\big(FK(\boldsymbol{q})\big) \subseteq \text{capsule}_\ell\big(FK(\boldsymbol{q}), r_\ell\big)$.*

Under assumption 1, the envelope (1) contains every link's true geometry over the entire box B. If $\text{AABB}_\ell(B) \cap O_j = \emptyset$ for all links ℓ and obstacles O_j, then $B \subseteq \mathcal{C}_{\text{free}}$. This is a conservative sufficient condition: intersection does *not* imply collision, only that the box cannot be certified at this granularity.

Computing the envelope via interval-FK. The standard DH transformation matrix $\mathbf{T}_i(\theta_i)$ maps frame i to frame $i{-}1$. Substituting interval trigonometric functions for joint-angle intervals yields an interval matrix $[\mathbf{T}_i]$. The forward kinematics to frame k is obtained by sequential interval matrix multiplication:

$$[\mathbf{T}^{0:k}] = [\mathbf{T}^{0:k-1}] \otimes [\mathbf{T}_k], \tag{2}$$

where $\otimes$ denotes interval matrix multiplication. The AABB for link ℓ is extracted from the translation vectors of $[\mathbf{T}^{0:\ell-1}]$ and $[\mathbf{T}^{0:\ell}]$ (representing the two endpoints), then inflated by r_ℓ.

Certified conservatism. By the inclusion-monotonicity property of interval arithmetic [9, Theorem 4.1], interval-FK provides a provably conservative AABB enclosure. Chain multiplication may introduce over-approximation due to the dependency problem (the *wrapping effect*). This affects the *tightness* of the enclosure but not its *soundness*.

Complexity. Certifying a single box requires computing the interval transform chain once (D interval matrix multiplications of fixed $4{\times}4$ size), then extracting L link AABBs. With prefix sharing, the total cost is $O(D + L)$.

B. Lifelong Envelope Cache Tree (LECT)

The LECT is a KD-tree over C-space that organizes and caches the AABB envelopes computed by interval-FK.

Structure. The root covers the full configuration space $\mathcal{C}$. Each internal node bisects one dimension. Each node v stores a C-space box B_v and its cached envelope vector $\mathcal{A}(B_v) = (\text{AABB}_1(B_v), \ldots, \text{AABB}_L(B_v))$.

Lazy growth. The LECT starts with a coarse root node covering $\mathcal{C}$. Nodes are split on demand when FFB queries (section IV-B) require finer granularity. A configurable maximum depth bounds the finest resolution. Deeper splits produce smaller C-space boxes with tighter AABB envelopes.

FK prefix reuse. When a LECT node splits along dimension d, the interval transform chain $[\mathbf{T}^{0:d-1}]$ for joints $1, \ldots, d{-}1$ is unchanged. Only $[\mathbf{T}^{d:D}]$ needs recomputation. This saves approximately 43% of computation on average (measured on IIWA14 7-DOF).

Lifelong reuse. Each node's AABB envelope is computed once by interval-FK and cached permanently. Because the envelope depends only on robot kinematics, a given C-space box's AABB never changes across scenes. The same LECT is reusable for the lifetime of a given robot.

Persistence. The LECT supports lightweight disk I/O: nodes are lazy-loaded on first access and incrementally appended via mmap. After warm start, the FK phase takes <1 ms. The bottleneck shifts entirely to collision detection.

C. Subdivision Enhancements

Both subdivision techniques below are orthogonally applicable within the interval-FK pipeline; they trade additional computation time for tighter certified envelopes.

Link subdivision. A long link's AABB can be excessively large. Splitting link ℓ into n equal segments along its local frame and computing AABBs independently tightens the envelope. Each sub-segment AABB is obtained by linearly interpolating the two endpoint positions already available from the FK chain; no additional FK evaluation is required. The only overhead is $n{-}1$ extra AABB extractions per link, which increases the number of AABB–obstacle tests in collision detection but adds negligible FK cost. Certification is preserved because each sub-segment AABB encloses the corresponding portion of the link geometry.

C-space box subdivision. Bisecting B along a dimension into B_1, B_2 and computing AABBs separately yields tighter enclosures. Recursive k-level subdivision produces 2^k sub-boxes. Under interval-FK, sub-boxes share prefix transforms and *certification is preserved* ($\bigcup_i \text{AABB}_\ell(B_i) \subseteq \text{AABB}_\ell(B)$ but tighter).

Both subdivision techniques compose orthogonally with each other. Thanks to interval-FK computation sharing (prefix-chain reuse), time overhead grows sub-linearly while tightness improves significantly.

IV. SafeBoxForest: Growth, Coarsening, and GCS Integration

Given a specific obstacle scene, SBF grows a forest of certified collision-free boxes on top of the LECT cache. The growth process resembles RRT: starting from seed configurations, boxes expand outward via face sampling. Greedy coarsening then merges fine boxes into larger regions. The resulting forest maps directly to a GCS graph.

A. RRT-like Forest Growth

The construction procedure is given in algorithm 1. Each box is obtained by calling *Find Free Box* (FFB, section IV-B),

Algorithm 1 SafeBoxForest Construction

Require: Robot R, obstacles $\mathcal{O}$, $\boldsymbol{q}_{\text{start}}$, $\boldsymbol{q}_{\text{goal}}$, max boxes $N_{\max}$, random seeds N_{rand}
Ensure: SafeBoxForest $\mathcal{F}$, GCS graph G
1: Initialize LECT T (load from disk if available)
2: $B_s \leftarrow \text{FFB}(T, \boldsymbol{q}_{\text{start}}, \mathcal{O})$; $B_g \leftarrow \text{FFB}(T, \boldsymbol{q}_{\text{goal}}, \mathcal{O})$
3: $Q \leftarrow \{B_s, B_g\}$; $\mathcal{F} \leftarrow \{B_s, B_g\}$
4: **while** $|\mathcal{F}| < N_{\max}$ **and** $Q \neq \emptyset$ **do**
5: $\quad B \leftarrow \text{dequeue}(Q)$
6: $\quad$ **for all** face $f \in \partial B$ **do**
7: $\quad\quad \boldsymbol{q}_{\text{seed}} \leftarrow \text{sample_outside_face}(f, \epsilon)$
8: $\quad\quad$ **if** goal_bias() **then**
9: $\quad\quad\quad \boldsymbol{q}_{\text{seed}} \leftarrow \text{perturb_toward}(\boldsymbol{q}_{\text{seed}}, \boldsymbol{q}_{\text{goal}})$
10: $\quad\quad$ **end if**
11: $\quad\quad B_{\text{new}} \leftarrow \text{FFB}(T, \boldsymbol{q}_{\text{seed}}, \mathcal{O})$
12: $\quad\quad$ **if** $B_{\text{new}} \neq \emptyset$ **and** $B_{\text{new}} \notin \mathcal{F}$ **then**
13: $\quad\quad\quad \mathcal{F} \leftarrow \mathcal{F} \cup \{B_{\text{new}}\}$; $Q \leftarrow Q \cup \{B_{\text{new}}\}$
14: $\quad\quad$ **end if**
15: $\quad$ **end for**
16: **end while**
17: {Free Interval Weighted Sampling}
18: **for** $i = 1$ **to** N_{rand} **do**
19: $\quad \boldsymbol{q}_{\text{seed}} \leftarrow \text{sample_free_interval_weighted}(\mathcal{F})$
20: $\quad B_{\text{new}} \leftarrow \text{FFB}(T, \boldsymbol{q}_{\text{seed}}, \mathcal{O})$
21: $\quad$ **if** $B_{\text{new}} \neq \emptyset$ **then**
22: $\quad\quad \mathcal{F} \leftarrow \mathcal{F} \cup \{B_{\text{new}}\}$
23: $\quad$ **end if**
24: **end for**
25: {Bridge Paving}
26: $\mathcal{C} \leftarrow \text{connected_components}(\mathcal{F})$
27: **for all** pair $(C_i, C_j) \in \mathcal{C} \times \mathcal{C}$ **do**
28: $\quad \text{bridge_paving}(T, C_i, C_j, \mathcal{O}, \mathcal{F})$
29: **end for**
30: $\mathcal{F} \leftarrow \text{greedy_coarsen}(\mathcal{F}, T, N_{\text{target}})$ {tree-cached hull check (section IV-D)}
31: $G \leftarrow \text{build_adjacency}(\mathcal{F})$
32: $\text{save_LECT}(T)$
33: **return** $\mathcal{F}, G$

which performs a top-down LECT search from a seed configuration.

Seed strategies. The growth process is analogous to RRT [13]: boxes expand outward from seed configurations, exploring free C-space incrementally. *BFS wavefront* (lines 4–14): starting from B_s and B_g, boxes expand outward via ϵ-offset face sampling. With probability $p_{\text{goal}} = 0.1$, the seed is perturbed toward $\boldsymbol{q}_{\text{goal}}$ to accelerate goal-directed expansion (analogous to RRT goal bias). *Free-interval weighting* (lines 16–19): sampling weighted by per-dimension uncovered interval length fills coverage gaps. *Bridge paving* (lines 21–23): isolated connected components are bridged in three steps. First, a *dry-run FFB* probes whether a candidate seed between two components can yield a free box. Then, an *overlap check* ensures the new box does not violate I1. Finally, a *formal FFB* inserts the box into $\mathcal{F}$. If direct bridging fails, an internal RRT-Connect [14] finds a collision-free path between the components. Additional boxes are paved along this path to restore connectivity.

B. Find Free Box (FFB)

FFB is the core subroutine called by each expansion step in algorithm 1. Given a seed configuration $\boldsymbol{q}_{\text{seed}}$, it performs a top-down search in the LECT:

1) If node B_v is collision-free $\rightarrow$ *promotion*: return B_v (largest collision-free ancestor).
2) If collision $\rightarrow$ *split*: bisect along the widest dimension, recurse into the child containing $\boldsymbol{q}_{\text{seed}}$.
3) At minimum-granularity leaf $\rightarrow$ return $\emptyset$ (mark as occupied).

FFB reuses the LECT's cached AABBs for collision checking. New nodes created during splitting are cached for future use.

Active split dimensions. Dimensions that do not affect any collision-relevant link geometry are automatically excluded, reducing unnecessary splits. For instance, if the last joint induces only a rotation about the link's symmetry axis so that the link AABB remains invariant, that dimension can be skipped.

Resolution completeness. If a collision-free path has ℓ_∞-clearance $\geq \delta$ and the LECT leaf size satisfies $\delta_{\min} \leq \delta/2$, FFB certifies a face-adjacent box chain covering the path (provided the wrapping effect does not cause false rejection at leaf resolution). SBF is therefore resolution-complete for paths of sufficient clearance; $\delta_{\min}$ controls the resolution.

C. Incremental Update

When the obstacle set changes from $\mathcal{O}$ to $\mathcal{O}'$, IRIS-family methods cannot pinpoint invalidated regions and must fully reconstruct at cost $O(N \cdot T_{\text{IRIS}})$. SBF can precisely identify and surgically repair affected boxes via a three-step *targeted regrowth* procedure:

1) **Invalidate**: construct a single-obstacle collision checker for each changed obstacle and test every box's cached link AABBs against it, collecting the invalidated set $\mathcal{F}_{\text{inv}}$. Cost: $O(|\mathcal{F}| \cdot L)$ per changed obstacle.
2) **Remove**: delete $\mathcal{F}_{\text{inv}}$ from $\mathcal{F}$ and the adjacency graph G; `remove_boxes` automatically cleans adjacency edges in both directions.
3) **Targeted regrow**: use the LECT's `sample_unoccupied_seed()` to draw seeds from depleted (unoccupied) tree regions. These regions are precisely the areas vacated by removed boxes. Each seed undergoes FFB, promotion, and `add_box_direct` ($O(N)$ incremental adjacency). This bypasses the full construction pipeline (wavefront, random fill, coarsening, global adjacency rebuild). The target count is $n_{\text{target}} = 2|\mathcal{F}_{\text{inv}}|$.

Because targeted regrow skips all heavyweight stages, its cost is proportional to the number of invalidated boxes rather than the total forest size. Typical single-obstacle perturbations require only $1/150$ of cold-rebuild time (table II).

D. Tree-Cached Greedy Coarsening

The BFS wavefront and random fill stages typically produce thousands of fine-grained boxes (*e.g.*, $\sim$2,000 for the IIWA14 combined scene). Greedy coarsening iteratively merges adjacent box pairs until a target count N_{target} is reached, preserving collision-safety certificates (algorithm 2).

Each hull candidate H is certified by a two-stage check. Stage 1 runs a fast full-box interval-FK test ($\sim\mu$s). Stage 2, invoked only when Stage 1 fails, traverses the LECT to verify only the gap region $H \setminus (A \cup B)$, reusing cached AABBs and skipping nodes already covered by A or B. An FK-call budget

Algorithm 2 Tree-Cached Greedy Coarsening

Require: Forest $\mathcal{F}$, collision checker, target N_{target}, HierAABBTree T, FK budget per round B_{fk}
Ensure: Coarsened forest $\mathcal{F}'$

```
repeat
    Rebuild face-adjacency pairs of F
    for each adjacent pair (A, B) do
        H ← bbox(A ∪ B);  s ← vol(H)/(vol(A) + vol(B))
        if s ≤ 50 then
            Add (s, A, B, H) to candidates
        end if
    end for
    Sort candidates by s ascending
    fk_count ← 0
    for each candidate (A, B, H) in order do
        if |F| ≤ N_target then
            break
        end if
        safe ← ¬ check_box(H) {Stage 1: fast interval-FK}
        if ¬safe and T ≠ null and fk_count < B_fk then
            safe ← T.check_hull_safe(H, A, B) {Stage 2}
        end if
        if safe then
            Merge A, B into H in F
        end if
    end for
until no merges this round or max rounds reached
return F
```

per round (default 2,000) prevents cost explosion. Tree-cached coarsening reduces box count from 1,042 to 457 (56% further reduction) while increasing coverage from 25.8% to 30.5%, all within 1 s.

E. GCS Graph Generation

The coarsened forest $\mathcal{F} = \{B_1, \ldots, B_N\}$ is mapped to a GCS graph $G = (V, E)$ as follows.

Nodes. Each box $B_i = [\boldsymbol{l}_i, \boldsymbol{u}_i]$ is an axis-aligned hyperrectangle, representable as a HPolyhedron with $2D$ halfplane constraints ($\boldsymbol{l}_i \leq \boldsymbol{q} \leq \boldsymbol{u}_i$). This representation requires no vertex enumeration or convex-hull computation.

Edges. Two boxes B_i, B_j are face-adjacent iff they share a $(D-1)$-dimensional face: one dimension d satisfies $u_i^d = l_j^d$ (or vice versa) and all other dimensions overlap. Each edge carries a continuity constraint ensuring the trajectory is continuous across the shared face.

Source and target. The boxes containing $\boldsymbol{q}_{\text{start}}$ and $\boldsymbol{q}_{\text{goal}}$ serve as source and target nodes. Boxes are non-overlapping ($\text{int}(B_i) \cap \text{int}(B_j) = \emptyset$ for $i \neq j$), so each query point belongs to at most one box.

V. Experiments

All experiments use a KUKA IIWA14 7-DOF manipulator (D=7, L=7) on Ubuntu 22.04 (Intel Core i7). SBF is implemented in C++ with pybind11 bindings; all GCS methods use Drake [18] with Mosek [19] as the SOCP backend. The test scene is the combined Marcucci benchmark [1] with 16 AABB obstacles (shelves, bins, and a table; fig. 2). Five query pairs form a cyclic tour through five milestone configurations. We compare **SBF-GCS** (ours) with **IRIS-NP-GCS**, both feeding into the same GCS-SOCP solver. C-IRIS [3] is excluded because a single region requires 2–5 min in 7-DOF settings. Both methods use 8 Marcucci milestone configurations as seeds; SBF repeats each setup 10 times (seed 0–9). To validate certification, we sample 10^4 configurations per box and check against FCL [20]; no collision is detected.

TABLE I
Region generation on the combined scene (SBF: mean±std, 10 seeds).

Method	# Regions	Coverage	Time	Speedup
SBF cold	892±59	(23.6 ± 1.8)%	**1.05 s**±0.27	**86**×
SBF warm	850±33	(24.5 ± 1.7)%	**0.73 s**±0.29	**123**×
IRIS-NP†	8	**60.0%**	89.95 s	1×

† 8 seed points, Drake IrisNp, `iteration_limit` = 10.
Regions = after greedy coarsening. Speedup vs. IRIS-NP.

A. Region Generation Efficiency

This experiment directly compares the core region-generation capability of SBF versus IRIS-NP, which is the current standard region generator for GCS planning.

Design. Both methods are run to completion on the combined scene with a reduced planning subspace (0.033% of the original C-space volume) derived from the milestone configurations with 0.3 rad margin. IRIS-NP uses 8 manually chosen seed configurations from [1] and Drake's `IrisNp` implementation with `iteration_limit` = 10. SBF uses the same 8 milestone configurations as BFS root seeds (box budget 1,000; random fill 200; $\delta_{\min}$=0.02 rad; coarsen target 200; split_depth 1; FK budget 2,000) and is evaluated under two conditions: *cold* (no LECT; tree built from scratch) and *warm* (LECT loaded from a previous run), demonstrating the cross-seed stability and cache reuse benefit. Coverage is estimated via Monte Carlo sampling (10^5 points evaluated purely within the 0.033% relevant subspace, independent RNG seed).

Analysis. IRIS-NP produces 8 high-volume polytopes (18.23 rad^7 total volume) achieving 60.0% free-space coverage, but requires 90 s (over 11 s per region). SBF cold generates ∼2,700 certified boxes and coarsens them to ∼892 in a total of 1.05 s (**86**× **faster** than IRIS-NP) with 23.6% coverage. SBF warm loads a previously cached LECT from disk. It achieves 24.5% coverage with ∼850 post-coarsening boxes in 0.73 s (**123**× **faster**), a **1.44**× **speedup** over cold start. The LECT warm start reduces build time by eliminating redundant FK computations. Cached AABBs enable the coarsening stage's tree-traversal to verify more hull candidates without recomputing forward kinematics. While IRIS-NP's individual polytopes are volumetrically larger, they lack collision-safety certificates; each SBF box carries a formal interval-FK certificate. The coverage gap reflects the conservatism of hyperrectangular envelopes (the wrapping effect). Importantly, this lower *global* coverage does not preclude effective planning. Figure 3 confirms that SBF's densely packed boxes concentrate coverage along planning-relevant corridors, achieving a 98% success rate with paths only 11% longer on average. Moreover, SBF's persistent cache enables instantaneous reuse across scenes (table II). IRIS-NP cannot provide this capability (table I).

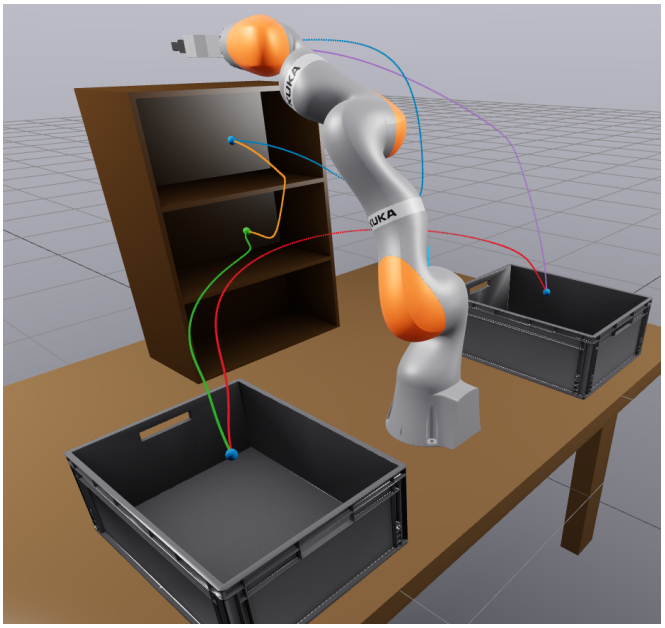


Fig. 2. 3-D workspace visualization of SBF-GCS planned trajectories on the combined scene (seed 0). Trajectory colors: Task 1 blue, 2 yellow, 3 green, 4 red, 5 purple.

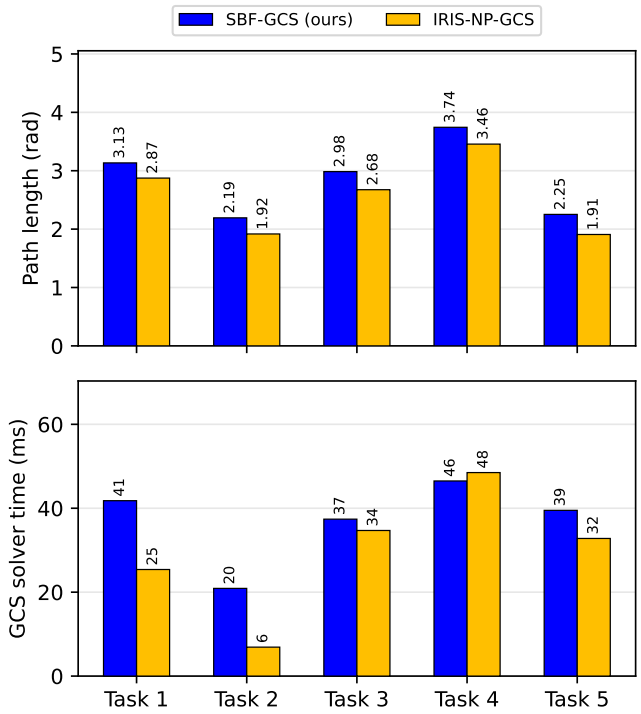


Fig. 3. Per-pair GCS planning results. Top: C-space path length (rad); bottom: GCS solver time (ms). SBF-GCS achieves 98% success (49/50) vs. IRIS-NP-GCS 100%. SBF: 10 seeds; IRIS-NP: 10 repeats.

Speed–coverage tradeoff. SBF's lower global coverage is not an inherent ceiling: increasing the box budget, subdivision depth, or random-fill rounds would raise coverage at the cost of longer build time. SBF deliberately prioritizes speed—completing region generation two orders of magnitude faster than IRIS-NP—while BFS wavefront seeding concentrates certified boxes along planning-relevant corridors, preserving a competitive success rate (fig. 3).

B. End-to-End GCS Planning Performance

This experiment evaluates whether SBF-generated regions translate into competitive downstream planning.

Metrics. Per-pair planning success rate (%); GCS solver time; path length (C-space L_2).

Analysis. SBF-GCS achieves a 98% overall success rate (49/50 query–seed combinations) versus 100% for IRIS-NP-GCS (fig. 3). Despite SBF's lower global coverage (24% vs. 60%, table I), the densely packed boxes concentrate certified regions along planning-relevant corridors, preserving connectivity for nearly all queries (fig. 2). SBF path lengths are on average 11% longer than IRIS-NP (mean 2.86 rad vs. 2.57 rad across all tasks), reflecting the finer granularity of hyperrectangular regions compared to large polytopes. The gap is smallest on Task 4 (8%, the hardest long-detour pair) and largest on Task 5 (18%). GCS solver times are comparable: SBF averages 37.2 ms vs. IRIS-NP 29.7 ms, and SBF is even slightly faster on Task 4 (46.5 ms vs. 48.5 ms) despite its larger graph (∼885 nodes vs. 8). Both methods' solver times remain in the tens-of-milliseconds range, negligible compared to region generation (SBF: ∼1 s; IRIS-NP: ∼90 s). The single failure arises from a connectivity gap in the box forest for one random seed; this is addressable by increasing the random-fill budget or bridge-paving effort.

TABLE II
INCREMENTAL UPDATE VS. WARM/COLD REBUILD AFTER 5 CM SINGLE-OBSTACLE PERTURBATION (100 TRIALS, 10 SEEDS × 10).

	Incremental	Warm	Cold
Time (median)	**7 ms**	727 ms	1,059 ms
Time (mean)	**11 ms**	775 ms	1,134 ms
Boxes (median)	916	854	854
Volume (rad^7, median)	5.82	8.98	8.98
Invalidated (med.)	90	—	—
Fraction of cold time	**1/150**	1/1.53	1
Vol. ratio (vs. cold)	0.65	1.0	1.0

C. Incremental Update

Starting from a fully constructed forest, we randomly select one bin or shelf obstacle and apply a 5 cm random translation. This simulates a typical single-object perturbation in manipulation tasks. Each seed undergoes 10 consecutive perturbation trials (100 trials total across 10 seeds). We compare three reconstruction strategies: *incremental* (targeted regrow: invalidation + `regrow`$(2n_{\text{inv}})$), *warm rebuild* (invalidation + full `build_multi` reusing the FK cache), and *cold rebuild* (fresh planner, no prior cache).

Analysis. A single-obstacle 5 cm perturbation invalidates ∼90 boxes (∼10% of the forest). Targeted regrow refills these gaps in **7 ms** (median). This is only **1/150** of the cold-rebuild time (7 ms vs. 1.06 s) and **1/104** of warm-rebuild time (7 ms vs. 0.73 s). The regrown forest contains slightly more boxes (916 vs. 854) because coarsening is skipped. The resulting volume is 65% of cold-rebuild volume. This is a

reasonable tradeoff given the two-orders-of-magnitude speed gain. Warm rebuild re-executes the full construction pipeline with FK-cache reuse. It achieves only $1.53\times$ speedup over cold, confirming that the bottleneck lies in the pipeline stages rather than FK computation. IRIS-NP provides no built-in mechanism for incremental updates. In practice, any scene change requires discarding and regrowing all regions from scratch ($\sim$90 s). SBF's 7 ms incremental update is four orders of magnitude faster than this full-rebuild baseline.

VI. Conclusion

We presented SafeBoxForest, a certified region generator for GCS motion planning backed by a lazy lifelong AABB cache. The LECT lazily maps C-space boxes to certified workspace AABB envelopes via interval-FK. Each envelope is computed on demand when first queried and cached permanently. Because envelopes depend only on robot kinematics, the cache is reusable for the robot's entire lifetime. Given a specific obstacle scene, SBF queries the LECT and grows a collision-free box forest via RRT-like wavefront expansion. Greedy coarsening merges fine boxes into hundreds of certified regions. The resulting forest maps directly to a GCS graph. SBF achieves competitive planning performance (98% success rate, paths only 11% longer than IRIS-NP on average) with region generation 86–$123\times$ faster than IRIS-NP. SBF also provides formal safety guarantees that IRIS-NP lacks. The LECT cache and targeted regrowth transform region generation from "rebuild every time" to "incremental maintenance." Regrowth requires only $1/150$ of cold-rebuild time (7 ms vs. 1.06 s) for single-obstacle updates. All safety guarantees are conditioned on assumption 1 and the AABB obstacle model; violations of either assumption void the certificate.

Limitations. Hyperrectangles yield lower per-region volume than polytopes (24% vs. 60% global coverage), though dense packing preserves a 98% success rate. The method assumes serial DH kinematics and AABB obstacles; for $D > 7$ the interval wrapping effect progressively loosens envelopes. Evaluation is limited to a single platform (KUKA IIWA14).

Future work. OBB envelopes or affine arithmetic could tighten enclosures and mitigate wrapping. Hybrid SBF–IRIS approaches, adaptive subdivision depth, and multi-core wavefront parallelization are promising extensions. Evaluation on additional manipulators and obstacle geometries remains an important direction.

References

[1] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science Robotics*, vol. 9, no. 84, p. eadf7843, 2024.

[2] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," *arXiv preprint arXiv:2303.14737*, 2023.

[3] P. Werner, T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Certified polyhedral decompositions of collision-free configuration space," *Int. J. Robotics Research*, 2024.

[4] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Proc. Workshop on the Algorithmic Foundations of Robotics (WAFR)*. Springer, 2015, pp. 109–124.

[5] P. Werner, T. Cohn, R. H. Jiang, T. Seyde, M. Simchowitz, R. Tedrake, and D. Rus, "Faster algorithms for growing collision-free convex polytopes in robot configuration space," in *Proc. Int. Symp. Robotics Research (ISRR)*, 2024, arXiv:2410.12649.

[6] J. T. Schwartz and M. Sharir, "On the "piano movers" problem. II. general techniques for computing topological properties of real algebraic manifolds," in *Advances in Applied Mathematics*. Academic Press, 1983, vol. 4, pp. 298–351.

[7] D. Zhu and J.-C. Latombe, "New heuristic algorithms for efficient hierarchical path planning," *IEEE Trans. Robotics and Automation*, vol. 7, no. 1, pp. 9–20, 1991.

[8] J.-M. Lien and N. M. Amato, "Approximate convex decomposition of polyhedra and its applications," *Computer Aided Geometric Design*, vol. 25, no. 7, pp. 503–522, 2008.

[9] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.

[10] L. Jaulin, M. Kieffer, O. Didrit, and É. Walter, *Applied Interval Analysis*. Springer, 2001.

[11] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robotics Research*, vol. 23, no. 3, pp. 221–235, 2004.

[12] D. Daney, N. Andreff, G. Chabert, and Y. Papegay, "Interval method for calibration of parallel robots: Vision-based experiments," *Mechanism and Machine Theory*, vol. 41, no. 8, pp. 929–944, 2006.

[13] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[14] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2000, pp. 995–1001.

[15] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[16] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.

[17] M. Phillips, B. J. Cohen, S. Chitta, and M. Likhachev, "E-graphs: Bootstrapping planning with experience graphs," in *Proc. Robotics: Science and Systems (RSS)*, 2012.

[18] R. Tedrake and the Drake Development Team, "Drake: Model-based design and verification for robotics," 2019. [Online]. Available: https://drake.mit.edu

[19] MOSEK ApS, "The MOSEK optimization toolbox," 2024. [Online]. Available: https://www.mosek.com

[20] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.